



Carrera **Desarrollo y Diseño Web**

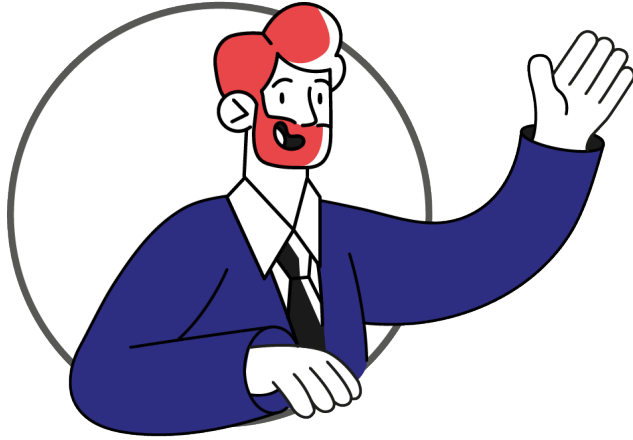
Experiencia 2 – Semana 5

**Mejorando la usabilidad del
sitio con JS**

Índice

Introducción a la semana.....	3
Resultado de aprendizaje.....	4
Conceptos relevantes.....	5
Preguntas activadoras.....	5
Actividad.....	6
Selección de elementos con CSS.....	7
Selección de elementos con querySelector y querySelectorAll.....	10
Ejemplos de uso práctico de querySelector y querySelectorAll.....	13
Modificar elementos.....	15
Ejemplo de uso de modificadores de elementos.....	17
Cierre de la semana.....	22
Referencias.....	23
Apuntes.....	24

Introducción a la semana



Durante esta semana, exploraremos nuevas formas de seleccionar elementos del Document Object Model (DOM). Comenzaremos con las funciones `querySelector` y `querySelectorAll`, que nos permiten identificar y acceder a elementos específicos en una página web utilizando selectores CSS, facilitando la manipulación dinámica del contenido.

Además, aprenderemos a crear nuevos elementos HTML mediante la función `createElement` y a insertar comentarios en el código con `createComment`. La función `appendChild` será clave para incorporar estos elementos recién creados al DOM, permitiéndonos construir y modificar la estructura de una página de manera dinámica. Asimismo, exploraremos el uso de `createTextNode` para agregar texto a nuestros elementos.

Dentro de esta guía encontraremos algunos conceptos relevantes:

- **Árbol**, que se refiere a la representación jerárquica y estructurada de un documento HTML o XML en forma de un árbol.
- **Relación padre/hijo**, que habla de la conexión jerárquica entre elementos HTML dentro de la estructura de un documento web.
- **Elemento**, que se refiere a una parte individual y específica de un documento HTML o XML, recordando que además, los atributos y el contenido asociados a cada etiqueta también forman parte de ese elemento.

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Utiliza elementos básicos de programación en JavaScript en sitio web, de acuerdo criterios de usabilidad y accesibilidad y los requerimientos del proyecto.

RA2. Programa en JavaScript, considerando el control de elementos BOM (browser) y DOM (documento) para un sitio web bajo criterios de usabilidad, accesibilidad y estéticos.

Indicadores de logro:

IL1. Utiliza correctamente los elementos básicos de programación como variables, arrays, operadores (if - else) y bucles, a fin de dar solución a los requerimientos del proyecto.

IL2. Utiliza el lenguaje JavaScript para programar funciones que permiten reutilizar código de manera eficiente.

IL3. Distingue las funciones y aplicaciones del BOM y DOM en el proceso de desarrollo de un sitio web con JavaScript.

IL4. Utiliza JavaScript para seleccionar y controlar elementos del DOM de manera dinámica, para crear un sitio web que responda a las acciones del usuario.

IL5. Manipula los elementos del DOM, a fin de mejorar la usabilidad, accesibilidad y estética del sitio web.

IL6. Integra la selección, manipulación y control del DOM con el uso de matrices, bucles y funciones.

Conceptos relevantes

	Árbol	Relación padre/hijo	Elemento
	querySelector y	querySelectorAll y	createElement
	createComment	createTextNode	appendChild

Preguntas activadoras

- ¿Sabías que la combinación de `querySelector` y `querySelectorAll` puede mejorar la eficiencia y flexibilidad al seleccionar elementos específicos en el DOM de una página web?
- ¿Conoces las ventajas de utilizar `createElement` y `appendChild`, en lugar de manipular directamente el HTML estático cuando se trata de agregar o modificar dinámicamente contenido en una página web?
- ¿Sabías que la manipulación del DOM en JavaScript, utilizando las técnicas mencionadas, puede contribuir a la creación de experiencias de usuario más interactivas y personalizadas en aplicaciones web?

Actividad

Descripción de la actividad

En esta actividad formativa tendrás que modificar en tiempo real el contenido de una página web, utilizando `querySelector`, `querySelectorAll`, `createElement`, `createComment`, `appendChild` y `createTextNode` en JavaScript.

Para realizar esta actividad, debes utilizar como punto de partida la página web que ya creaste en la semana 4. Por tanto, no necesitas empezar desde cero con una nueva página HTML, sino que modificarás y mejorarás una página existente.

Selección de elementos con CSS

Los selectores de CSS son patrones utilizados para seleccionar los elementos del DOM. Son fundamentales para aplicar estilos CSS y también son utilizados por métodos de JavaScript como `querySelector` y `querySelectorAll`.

Para poder sacar el máximo provecho a estos métodos, recordemos los patrones que usan los selectores CSS.

Selector de tipo:

Selecciona todos los elementos de un tipo específico.

Figura 1

Ejemplo de selector de tipo.

```
p {  
  color: blue;  
}
```

Selector de clase:

Selecciona todos los elementos que tienen una clase específica.

Figura 2

Ejemplo de selector de clase.

```
.destacado {  
  font-weight: bold;  
}
```

Selector de ID:

Selecciona un único elemento con un ID específico.

Figura 3

Ejemplo de selector de ID.

```
#encabezado {  
  font-size: 24px;  
}
```

Selector de atributo:

Selecciona elementos con un atributo específico.

Figura 4

Ejemplo de selector de atributo.

```
input[type="text"] {  
  border: 1px solid #ccc;  
}
```

Selector de descendencia:

Selecciona elementos que son descendientes de otro elemento.

Figura 5

Ejemplo de selector de descendencia.

```
div p {  
  margin-bottom: 10px;  
}
```


Selector de clasificación (combinadores):

Selecciona elementos que cumplen múltiples condiciones.

Figura 6

Ejemplo de selector de clasificación.

```
article h2 {  
  color: green;  
}
```

Selector de pseudo-clase:

Selecciona elementos en un estado específico.

Figura 7

Ejemplo de selector de pseudo-clase.

```
a:hover {  
  text-decoration: underline;  
}
```

Selector de pseudo-elemento:

Selecciona partes específicas de un elemento.

Figura 8

Ejemplo de selector de pseudo-elemento.

```
p::first-line {  
  font-weight: bold;  
}
```

Combinación de selectores

Estos son solo algunos ejemplos básicos de selectores CSS. Aunque existen varios tipos de selectores, cada uno con su propósito específico, también puedes combinar selectores para ser más específico en tus reglas de estilo. Por ejemplo, el selector `div p` es un tipo de selector de descendencia. Por tanto, seleccionará todos los elementos `<p>` que se encuentren dentro de un elemento `<div>`. Esta combinación permite aplicar estilos específicos a los elementos `<p>` que solo están dentro de los `<div>`, sin afectar a otros elementos `<p>` que puedan estar en otras partes de la página.

Selección de elementos con `querySelector` y `querySelectorAll`

Como sabemos, JavaScript nos facilita 3 métodos para seleccionar elementos del DOM: `getElementById`, `getElementsByTagName` y `getElementsByClassName`. Sin embargo, existen dos métodos más: **`querySelector`** y **`querySelectorAll`**, los que se basan exclusivamente en usar los selectores CSS.

Estos métodos son similares a CSS, en el sentido de que utilizan selectores CSS para especificar qué elementos elegir. Al utilizar **`querySelector`** y **`querySelectorAll`**, puedes aplicar las mismas convenciones de selección que usarías en una hoja de estilo CSS.

Método `querySelector`:

Devuelve **el primer elemento** que coincide con el selector CSS especificado. Es útil cuando necesitas encontrar un elemento único o el primer elemento de un tipo particular.

Figura 9

Ejemplo de `querySelector`

```
let miElemento = document.querySelector("#miElemento");
```

Método `querySelectorAll`

Devuelve **todos los elementos** que coinciden con el selector CSS dado en forma de `NodeList`. Puedes iterar sobre esta lista para trabajar con cada elemento por separado.

Figura 10

Ejemplo de `querySelectorAll`

```
let elementos = document.querySelectorAll(".claseElemento");
```

Ejemplo de búsqueda

A continuación, revisemos un ejemplo de cómo se pueden usar los métodos `querySelector` y `querySelectorAll` para buscar y seleccionar elementos dentro de un documento HTML utilizando JavaScript.

La figura 11 muestra un fragmento de código HTML que define una lista desordenada (`<ul>`) con tres elementos de lista (`<li>`). Cada elemento de la lista tiene asignada la misma clase "item".

Figura 11

HTML con listado de elementos

```
<ul>
  <li class="item">Elemento 1</li>
  <li class="item">Elemento 2</li>
  <li class="item">Elemento 3</li>
</ul>
```

¿Cómo utilizar JavaScript para interactuar con la lista definida en el HTML?

Puedes seleccionar y manipular estos elementos en JavaScript de la siguiente manera: con el método `document.querySelector` puedes seleccionar el primer elemento `<li>` con la clase "item" del DOM. Después de seleccionar el primer elemento `<li>` con `querySelector`, se accede a su propiedad de estilo y se cambia el color del texto a rojo mediante `primerElemento.style.color = "red";`. Esto es útil cuando se quiere obtener una referencia rápida a un solo elemento para realizar operaciones como cambiar estilos, escuchar eventos, o manipular el DOM de otras maneras.

Mientras que utilizar el método `document.querySelectorAll`, permite seleccionar todos los elementos `<li>` con la clase "item" y los devuelve como una `NodeList`. Se itera sobre cada elemento de esta lista con `todosLosElementos.forEach(...)`, y dentro de esta iteración, se cambia el peso de la fuente a negrita para cada elemento `<li>` mediante `elemento.style.fontWeight = "bold";`.

Se utiliza para trabajar con múltiples elementos y aplicarles cambios de manera eficiente, como en bucles o cuando se utiliza el método para iterar sobre todos los elementos seleccionados y modificar sus estilos o atributos.

Figura 12

Ejemplos de selección con selectores de CSS

```
// Seleccionar el primer elemento li con la clase "item"
let primerElemento = document.querySelector("li.item");

// Seleccionar todos los elementos li con la clase "item"
let todosLosElementos = document.querySelectorAll("li.item");

// Aplicar estilos a los elementos seleccionados
primerElemento.style.color = "red";

// Iterar sobre la NodeList y modificar cada elemento
todosLosElementos.forEach(elemento => {
  elemento.style.fontWeight = "bold";
});
```

Como ves, en este ejemplo, `querySelector` y `querySelectorAll` son utilizados para seleccionar elementos basándose en reglas de selección CSS, y luego se manipulan los estilos de estos elementos en JavaScript.

Ejemplos de uso práctico de `querySelector` y `querySelectorAll`

Después de revisar los diferentes tipos de selectores CSS, los métodos `querySelector` y `querySelectorAll` y un ejemplo de búsqueda con ellos, revisemos ejemplos sobre cómo utilizar cada uno de estos métodos en un contexto práctico. En el caso de `querySelector`, veremos cómo utilizarlo para seleccionar un solo elemento y para `querySelectorAll`, revisaremos cómo se usa para seleccionar varios elementos que cumplan un criterio determinado.

Uso de `querySelector`

Supongamos que tienes una página web con una lista de productos y quieres resaltar el primer producto de alguna manera. Aquí puedes usar `querySelector` para seleccionar el primer elemento de la lista y aplicarle un estilo específico.

Para comprender esto con más claridad, observa la figura 13. Aquí se muestra un fragmento de código HTML que muestra una lista desordenada con el ID `listaProductos`. Esta lista contiene tres elementos de lista (`<li>`), cada uno con un producto numerado.

Figura 13

HTML del ejemplo de uso de `querySelector`

```
<ul id="listaProductos">
  <li>Producto 1</li>
  <li>Producto 2</li>
  <li>Producto 3</li>
</ul>
```

Para seleccionar el primer elemento de la lista () dentro del contenedor listaProductos, se utiliza el método `querySelector`. En este caso, y tal como se visualiza en la figura 14, el selector utilizado es `#listaProductos li:first-child`.

Una vez que se ha seleccionado el primer elemento de la lista, se le aplica un estilo directamente a través de JavaScript para resaltarlo. En este ejemplo, se le aplica un estilo CSS que cambia el peso de la fuente a negrita (`font-weight: bold`).

Figura 14

JavaScript del ejemplo de uso de `querySelector`

```
// Seleccionar el primer elemento de la lista
let primerProducto = document.querySelector("#listaProductos li:first-child");

// Aplicar un estilo al primer producto
primerProducto.style.fontWeight = "bold";
```

Ejemplo de uso de `querySelectorAll`

Imagina que tienes una galería de imágenes y quieres aplicar un efecto de sombra a todas las imágenes con una clase específica. Aquí, `querySelectorAll` permite seleccionar todos los elementos que cumplen con la condición.

La figura 15 muestra el HTML para una galería de imágenes. Vemos que el elemento `div` tiene una clase llamada `galería` y que sirve de contenedor para tres elementos `<img>`, dos de los cuales tienen asignada la clase `imagenDestacada`.

Figura 15

HTML del ejemplo de uso de `querySelectorAll`

```
<div class="galería">
  
  
  
</div>
```

En este caso, **querySelectorAll** se utiliza para seleccionar todos los elementos con la clase **"imagenDestacada"** dentro del contenedor con la clase **"galeria"**. Luego, se itera sobre cada elemento en la NodeList utilizando el método **forEach**. En cada iteración, se toma un elemento de la lista y se aplica una función de flecha (**imagen => ...**). La función toma un parámetro (**imagen**), y su cuerpo establece el estilo **boxShadow** para ese elemento en particular. Por tanto, en cada iteración se aplica un efecto de sombra a la imagen correspondiente. A su vez, todas las imágenes con la clase **imagenDestacada** recibirán este estilo, diferenciándolas de las que no tienen esta clase.

Figura 16

*JavaScript del ejemplo de uso de **querySelectorAll***

```
// Seleccionar todas las imágenes con la clase "imagenDestacada"
let imagenesDestacadas = document.querySelectorAll(".galeria .imagenDestacada");

// Aplicar un efecto de sombra a todas las imágenes destacadas
imagenesDestacadas.forEach(imagen => {
  imagen.style.boxShadow = "3px 3px 5px #888888";
});
```

Modificar elementos

Existen cuatro funciones clave que nos facilitan la manipulación del contenido de nuestras páginas web. Estos son **createElement**, **createComment**, **appendChild**, y **createTextNode**. Estas funciones proporcionan una manera programática de crear elementos HTML, comentarios, insertar nodos de texto, y estructurar el árbol DOM de manera efectiva, permitiendo a los desarrolladores modificar y construir contenido de manera dinámica.

createElement:

Se utiliza para crear un nuevo elemento HTML. Puedes especificar el tipo de elemento que deseas crear, como "div", "p", "span", entre otros.

Utilizar esta función permite la creación de nuevos elementos en el documento que luego pueden ser insertados en el árbol DOM en la ubicación deseada.

Figura 17

Ejemplo de createElement

```
let nuevoElemento = document.createElement("div");
```

createComment

Permite la creación de comentarios en el código HTML. Los comentarios son útiles para añadir anotaciones o explicaciones en el código fuente sin afectar el rendimiento del sitio web.

Figura 18

Ejemplo de createComment

```
let nuevoComentario = document.createComment("Este es un comentario descriptivo");
```

appendChild

Se utiliza para agregar un nuevo elemento como hijo de otro elemento en el DOM. Puedes construir la estructura del DOM conectando elementos padre e hijo. Por ello, es esencial para construir y modificar la estructura del DOM.

Por ejemplo, en la figura 19, se utiliza el método appendChild sobre el contenedor. Esto toma un nodo como argumento (en este caso, nuevoElemento) y lo añade al final de la lista de hijos del elemento contenedor. Es decir, nuevoElemento será insertado dentro del elemento con ID miContenedor en la estructura de la página.

Figura 19

Ejemplo appendChild

```
let contenedor = document.getElementById("miContenedor");  
contenedor.appendChild(nuevoElemento);
```

createTextNode

Los nodos de texto son los que contienen el texto que se muestra en los elementos HTML. Esta función crea un nodo de texto que puede ser insertado en un elemento. Este nodo de texto puede contener cualquier cadena de texto que desees mostrar en tu página.

Figura 20

Ejemplo createTextNode

```
let texto = document.createTextNode("Hola, este es un texto");
```

Ejemplo de uso de modificadores de elementos

A continuación, revisaremos un ejemplo de cómo el JavaScript puede manipular el DOM para actualizar dinámicamente el contenido de una página web. Utilizaremos createElement, createComment, appendChild, y createTextNode para crear una estructura de comentarios básica y agregarla al DOM.

Supongamos que estás desarrollando una aplicación de comentarios en una página web y deseas permitir a los usuarios agregar comentarios.

Tal como vemos en la figura 21, la estructura HTML sobre el cual la funcionalidad de comentarios se construirá, tiene los siguientes elementos:

- Un **contenedor <div>** con el ID **comentarios-container** que alojará los comentarios agregados.
- Un **<textarea>** con el ID **nuevo-comentario** para que los usuarios escriban sus comentarios.
- Un **<button>** con un evento **onclick** que llama a la función **agregarComentario()** cuando se hace clic en él.
- Una referencia a un archivo JavaScript externo **app.js** donde se encuentra la función **agregarComentario()**. El archivo **app.js** es un archivo externo de JavaScript que se utiliza para organizar y contener el código JavaScript que añade funcionalidad interactiva a una página web

Figura 21

HTML usado en el ejemplo de agregar comentarios

```
<!DOCTYPE html>
<html lang="es-cl">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Comentarios</title>
</head>

<body>
  <div id="comentarios-container"></div>
  <textarea id="nuevo-comentario" placeholder="Añade un comentario..."></textarea>
  <button onclick="agregarComentario()">Publicar Comentario</button>

  <script src="app.js"></script>
</body>

</html>
```

Como se observó en el código HTML de la figura 21, cuando el usuario escribe algo en el textarea y luego, hace clic en el botón "Publicar Comentario", se desencadena un evento debido al atributo onclick="agregarComentario()" en el botón. Esto llama al archivo app.js que contiene el código JavaScript donde se define la función agregarComentario() que es llamada cuando el usuario hace clic en el botón.

A continuación, revisemos la función agregarComentario() definida en app.js.

Figura 22

Función agregarComentario() definida en app.js

```
function agregarComentario() {  
    // Obtener el contenido del nuevo comentario desde el textarea  
    const nuevoComentarioTexto = document.getElementById("nuevo-comentario").value;  
  
    // Crear un nuevo comentario HTML  
    const nuevoComentario = document.createElement("div");  
  
    // Crear un nodo de texto con el contenido del comentario  
    const textoComentario = document.createTextNode(nuevoComentarioTexto);  
  
    // Crear un comentario HTML  
    const comentarioHTML = document.createComment("Comentario creado dinámicamente");  
  
    // Agregar el nodo de texto como hijo del nuevo comentario  
    nuevoComentario.appendChild(textoComentario);  
  
    // Agregar el comentario HTML como hijo del nuevo comentario  
    nuevoComentario.appendChild(comentarioHTML);  
  
    // Obtener el contenedor de comentarios existente en el DOM  
    const contenedorComentarios = document.getElementById("comentarios-container");  
  
    // Agregar el nuevo comentario al contenedor de comentarios  
    contenedorComentarios.appendChild(nuevoComentario);  
  
    // Limpiar el contenido del textarea  
    document.getElementById("nuevo-comentario").value = "";  
}
```

Para comprender con claridad el funcionamiento de la función `agregarComentario()`, revisemos una breve explicación de este proceso:

- **Obtener el texto del comentario: `getElementById("nuevo-comentario")`**
Para comenzar, obtenemos el texto desde el input que tiene el identificado ID “nuevo-comentario”.
- **Crear un nuevo contenedor para el comentario: `createElement("div")`**
Luego, creamos un nuevo elemento **div** que representará un comentario en la página. Por tanto, será utilizado para contener el comentario del usuario.
- **Crear un nodo de texto con el comentario: `createTextNode(nuevoComentarioTexto)`**
Creamos un **nodo** de texto con el contenido del comentario ingresado por el usuario en el **textarea**.
- **Crear un comentario HTML: `createComment("Comentario creado dinámicamente")`**
También creamos un comentario HTML como parte del comentario. Este comentario no es visible en la página, pero podría ser útil para propósitos de desarrollo o anotaciones.
- **Agregar el nodo de texto y el comentario HTML al contenedor del comentario: `appendChild`**
Agregamos el nodo de texto y el comentario HTML **como hijos del nuevo elemento div** que representa el comentario. Ahora, el div tiene contenido: el comentario del usuario.
- **Obtener el contenedor de comentarios del DOM: `getElementById("comentarios-container")`**
Para ello, la función busca en el DOM el elemento con el ID `comentarios-container`, que es el lugar donde se agregarán todos los comentarios.

- **Agregar el nuevo comentario al contenedor de comentarios:**

appendChild(nuevoComentario)

El div que contiene el nuevo comentario se añade como un hijo al contenedor de comentarios. Esto hace que el comentario sea visible en la página web.

- **Limpiar el área de texto: getElementById("nuevo-comentario")**

Finalmente, limpiamos el contenido de “nuevo-comentario” al establecer su valor (value) como vacío. De este modo, el usuario podrá escribir otro comentario si lo desea.

Como vimos en este ejemplo, al presionar el botón "Publicar Comentario", se toma el contenido del textarea, se crea un nuevo elemento div que contiene el comentario y se agrega al contenedor de comentarios existente en el DOM. Gracias a la función agregarComentario(), los usuarios podrán añadir comentarios y verlos reflejados en la página sin necesidad de recargarla.

Cierre de la semana

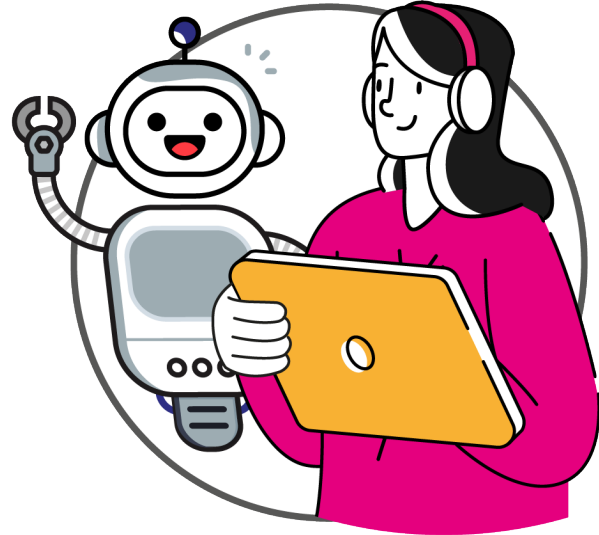
Al final de esta semana has conocido nuevas formas de buscar elementos del DOM, tales como `querySelector`, `querySelectorAll`. También conociste funciones clave como `createElement`, `createComment`, `appendChild`, y `createTextNode` para crear nuevos nodos en el árbol de DOM, de forma de poder agregar elementos que no existían previamente.

Con `createElement` puedes generar nuevos elementos HTML, como un `div` que podría

contener un comentario de usuario. `createTextNode`, por su parte, permite crear el contenido textual de ese comentario, mientras que `createComment` es útil para añadir notas dentro del código que no serán visibles en la interfaz de usuario, pero son valiosas para el desarrollo. Una vez creados, estos elementos pueden ser estructurados dentro del DOM mediante `appendChild`, que inserta un nodo hijo dentro de un nodo padre ya existente en el árbol. Es así como aprendiste a crear nuevos nodos en el árbol de DOM, de forma de poder agregar elementos que no existían previamente.

A partir de ahora, podrás agregar contenido que antes no existía, en respuesta a la interacción del usuario, como podría ser un comentario en un blog o una entrada en un foro, logrando así una experiencia de usuario rica y dinámica.

¡Recuerda poner en práctica todo lo aprendido para fortalecer tu aprendizaje en JavaScript!



Referencias

- Ramírez, L. (Comp.). (2023) *JavaScript / 5 cosas que hay que saber*
<http://js.luisfel.cl>
- Mozilla Web Docs, Avanzado, Chile (s.f.). *Guía de JavaScript*
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Introducci%C3%B3n>

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



DuocUC[®] ONLINE

Facilitador disciplinar: Claudio Navarro

Asesora par: Paola Véliz

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.